

CS 30700: Sprint 3 Retrospective

BoilerSpace

Team 3: Aagam Bapna, Sheehan Dandapat, Rohan Potta, Nathan Wan, Jed Macapinlac, Bidit Acharyya

What went well?

- **User Story 7:** As a user, I want to set my public/private profile visibility so that I control what others see. This is key for any social networking web app that wants to let users have control of who is allowed to view their profiles. Being able to successfully implement this user story meant that user now can have piece of mind when logging into our web app and understanding that their visibility is in their hand while still enjoying all the features that this app offers.
- **User Story 14:** As a user, I want to see my total study time for this week, so that I can track my productivity. For us, implementing this user story was key to rounding out the user experience for this web app. This meant that with not only being able to find suitable study partners, but also students being able to hold themselves accountable for the studying that they plan to do for over the week. By being able to set out weekly goals for themselves while the web app doing all the heavy lifting (being able to track study time as the checks into a room), it enables the web app to have a homey feel.
- **User Story 53:** As a student, I want to rate and post reviews on study areas so others can find the best spots. This user story is valuable when considering the community and discovery features within our web-app, as students benefit from shared, firsthand insights about which spaces actually support productive studying rather than relying on guesswork or word of mouth. By achieving this, we were able to empower students to make informed decisions about where to study, while also giving them a voice to contribute to a growing knowledge base that helps the broader student community find spots that fit their needs.
- **User Story 27:** As a student, I want to filter the map for quiet zones specifically so I can find a place for deep focus rather than group work. This user story is essential when considering the productivity and personalization features within

our web app, as students have different study needs depending on the task at hand and deserve tools that help them quickly find environments that match those needs. By achieving this, we were able to give students more control over their study experience, allowing them to cut through the noise of general location listings and zero in on spaces that support deep, focused work when that's what their day calls for.

- **User Story 38:** As an admin, I want to be able to broadcast announcements so users can receive platform updates. This user story is important when considering the communication and platform management features within our web-app, as admins need a reliable way to keep the user base informed about updates, changes, or important information that affects their experience. By achieving this, we were able to establish a direct line of communication between the platform and its users, ensuring that important announcements reach everyone in a timely and consistent manner rather than relying on scattered or external channels.
- **User Story 86:** As a user, I want to view my own and other users' public profiles so I can learn more about study partners before connecting with them. This is key as people want to learn about each other and display their self interests before committing to meeting and studying with others. This feature enables students to have a more open view of their classmates and have a better time matching with suitable ones.
- **User Story 87:** As a user, I want to see typing indicators, read receipts, and online/offline presence in my DMs so I know when others are active and have seen my messages. This user story is important when considering the real time communication features within our web app, as users benefit from knowing whether their messages have been seen and whether someone is actively responding rather than being left guessing. By achieving this, we were able to make conversations feel more natural and connected, giving users the live feedback they expect from a modern messaging experience.
- **User Story 88:** As a user, I want to approve DM requests from people I am not friends with so strangers cannot message me without permission. This user story

is important when considering the privacy and safety features within our web app, as users deserve control over who can reach them directly rather than having their inbox open to anyone on the platform. By achieving this, we were able to give users a clear boundary between their main inbox and unknown senders, making the messaging experience feel safer and more intentional.

- **User Story 89:** As a user, I want to search through all my messages globally and within a specific conversation so I can find anything without scrolling. This user story is important when considering the usability and navigation features within our web app, as users accumulate conversations over time and need a fast way to locate specific information without manually scrolling through long message histories. By achieving this, we were able to give users a powerful search tool that works both across all their conversations and within a single chat, saving time and making the app much more practical for everyday use.
- **User Story 72:** As a club organizer, I want to assign roles and create custom positions for members so that responsibilities are clearly defined and access is controlled for each club. This user story is valuable because growing student organizations need structure, and without clear role boundaries, tasks become unclear and permissions can become inconsistent or risky. By completing this, we enabled clubs to grow their operations more effectively through customizable positions and permissions that reflect each club's unique leadership model.
- **User Story 74:** As a club organizer, I want to schedule recurring events and view them in a calendar so that members can plan attendance in advance and easily see all events for the month. This user story is valuable because many club activities follow repeating schedules, and manually creating individual events is time-consuming and error-prone for organizers. By delivering this, we reduced event management overhead while improving discoverability for members through a visual monthly calendar that presents both one-time and recurring events in a clear, actionable format.
- **User Story 83:** As a student, I want to react to messages with simple reactions so that I can acknowledge messages without replying. This user story is valuable because not every conversation requires a full text response, and lightweight

interaction helps communication feel faster and less intrusive. By implementing this, we improved engagement and clarity in direct messaging by allowing students to quickly confirm, support, or respond emotionally to messages while keeping chats clean and reducing unnecessary message noise.

- **User Story 11:** As a user, I want to manage my notification preferences from my profile, so that I can control how and when I'm contacted. This user story is valuable when considering how easy it is for a multi-feature platform like BoilerSpace to overwhelm its users with notification noise such as alerts for messages, study sessions, club events, and announcements all stack up quickly, and without granular control students will either silence everything or stop opening the app altogether. By achieving this, we were able to let each user define exactly which signals deserve their attention and which ones don't, while also giving them a single Focus Mode toggle to pause every category during deep study without losing the underlying preferences they had configured.
- **User Story 48:** As a student, I want to know certain exam dates for certain classes so that I can prioritize my study sessions effectively. This user story is valuable when considering how scattered exam information typically is across syllabi, Brightspace pages, and verbal announcements, since students rarely have a single place to see what's coming up across all their courses at once. By achieving this, we were able to turn each course page into a deadline-aware planning surface that surfaces upcoming exams chronologically and visually flags any exam within the next week, while also letting students crowdsource exam dates for the courses they share so the entire enrolled cohort benefits from one person filling them in.
- **User Story 59:** As a student, I want to bookmark helpful AI responses, so that I can easily return to them later. This user story is valuable when considering how the AI Q&A feature, while powerful, is otherwise not as useful since once a question is answered and the modal closes, that explanation is gone unless the student re-asks the exact same question, which discourages students from leaning on the AI for anything they want to revisit. By achieving this, we were able to transform the AI assistant from a one-off lookup tool into a persistent

personal study reference where every helpful response can be saved, scrolled through, and re-read later, while also giving students a clean dedicated page to manage and curate the explanations that matter most for their courses.

- **User Story 20:** As a student, I want to receive notifications when new content is uploaded to courses, sessions, or events that I follow so that I stay up to date without manually checking the platform. This user story is valuable as it lets users know when new notes are uploaded in their classes or new sessions are added in organizations they are involved in. We were able to make it so that when a student receives a notification, they are alerted about it by showing the number of unread notifications. When they click on the notification, it takes them to the post that the notification was about.
- **User Story 21:** As a student, I want to temporarily mute all notifications, so that I can focus without interruptions. This user story is valuable as it allows students to control the notifications they receive in the event that they want to keep their inbox clean. All we did was remove the notification counter over the inbox icon, so students can still view their notifications in their inbox if they check themselves.
- **User Story 60:** As a student, I want the AI to help create a study plan and schedule so I can organize my time efficiently. This user story is a core part of the app as it helps students create plans to study for exams in their courses. It allows students to create plans structured around their existing commitments, and the generated plans are also able to be edited by users in case they want to make updates.

What did not go well?

Overall, Sprint 3 could have gone better in a couple of areas.

First, some of our user stories were not as ready to present as we would have liked. A few of our acceptance criteria were blocked or could not be demoed during the sprint review, which meant we were not able to show the full picture of what we built. Going

forward we want to make sure each user story is in a state where all of its acceptance criteria can actually be shown before we bring it to a demo.

Second, our testing coverage could be more transparent. Right now it is not always clear which unit tests map to which acceptance criteria, making it harder to verify that everything is properly covered. We want to improve this by making the connection between each test and the acceptance criteria it targets more visible and easy to follow.

How should we improve?

1. Strengthen test development by ensuring every acceptance criterion is covered by a unit test.

We ran into recurring issues where our testing approach didn't fully validate whether features actually met their requirements. Demonstration style tests were useful for showing that the happy path worked, but they didn't comprehensively cover edge cases, error states, or the full set of acceptance criteria tied to each story. This meant bugs and gaps in functionality sometimes slipped through to later stages, where they were more expensive to catch and fix. In some cases, we'd mark work as complete based on a successful demo, only to discover later that a specific acceptance criterion hadn't actually been verified in code.

Going forward, we should treat acceptance criteria as the foundation of our test plan, with each criterion mapped to at least one corresponding unit test. Demonstration tests can still play a role for integration level validation and grader visibility, but they shouldn't be the primary mechanism for confirming that requirements are met. By tying unit tests directly to acceptance criteria, we strengthen the project's reliability, make regressions easier to catch, and give ourselves a clearer definition of "done" that doesn't depend solely on a live demo working on a given day.

2. Be more intentional about pulling the latest main before demos to ensure consistency across presenters.

During our demos, we ran into situations where different team members were presenting from branches that weren't fully up to date with main. This created visible inconsistencies when

switching between presenters mid-demo, features that worked in one person's environment behaved differently and some UI elements didn't match. This undermined the polish of the demo and made it harder for our grader to follow along, since the product appeared less stable or cohesive than it actually was. It could also occasionally lead to confusion about which version of a feature was the "real" one, especially when older branches still had outdated implementations or stale data.

Going forward, we should establish a clear pre-demo checklist that includes pulling the latest main, resolving any merge conflicts, and verifying that the local build reflects the current state of the project. Ideally, every presenter should be working from the same commit, or at minimum from branches that have all been recently rebased onto main. We could also consider doing a brief team sync or dry run shortly before the demo to confirm everyone's environment is aligned. Taking these steps will help us present a unified, accurate version of the product, reduce mid-demo surprises, and make our progress easier for our grader to evaluate.

3. Prepare realistic test data and demo scenarios in advance to avoid disrupting the flow of presentations.

During our demos, we ran into moments where presenters had to manually create or paste in content on the fly to demonstrate certain features. For example, when showing the admin broadcast functionality, Jed had to take pre-written broadcast messages and paste them into the app during the demo itself. While the feature worked as intended, the act of stopping to copy, paste, and set up the input mid-demo disrupted the natural flow of the presentation and drew attention away from the functionality we were actually trying to showcase. This pattern came up in a few different places, and each time it made the demo feel less polished and harder for our grader to follow, since they had to wait through setup steps rather than seeing the feature in action immediately.

Going forward, we should invest more effort in preparing realistic test data and demo scenarios ahead of time, so that features can be shown in a state that's ready to demonstrate. This could include seeding the database with representative content before the demo, creating dedicated demo accounts with pre-configured data, or scripting setup steps so they can be run quickly and consistently before each presentation. Where live input is genuinely part of what we want to show, like an admin actually typing a broadcast, we should plan that intentionally rather than having it happen as a side effect of missing setup. Taking this approach will let presenters focus

on walking our grader through the feature itself, keep the demo flowing smoothly, and make the product feel more complete and professional during reviews.

4. Ensure every team member has access to all required API keys and credentials needed to run tests reliably.

We ran into a situation where only a select few team members had the full set of API keys and credentials needed to run our unit tests cleanly. The rest of us could technically still run the tests, but we'd see warnings or partial failures tied to missing keys, which made the test results less trustworthy and harder to act on. This created a bottleneck where only certain people could confidently verify whether tests were actually passing, and it made it harder for the rest of the team to debug failures locally or validate their own changes before pushing. It also meant that test output had to be interpreted differently depending on who was running it, since warnings from missing keys could mask or be confused with real failures.

Going forward, we should make sure that every team member has access to the full set of keys and credentials needed to run the test suite without warnings or skipped checks. This could be handled by maintaining a shared, secure location for development keys (such as a password manager or secrets vault), documenting clearly in the README which keys are required and how to obtain them, and keeping `.env.example` updated so it's obvious what's missing in a given environment. For sensitive production credentials, we can use scoped development or test keys instead, so no one needs access to anything they shouldn't have. Making sure everyone can run the full test suite cleanly will give us more consistent results across the team, reduce the bottleneck on a few key people, and make our testing process more reliable overall.

5. Prepare backup materials for acceptance criteria that involve unavoidable wait times during demos.

Some of our acceptance criteria involved features with inherent delays, such as AI generation, processing tasks, or other operations that simply take time to complete. During the demo, these moments created awkward pauses where we had to wait for the system to respond, which slowed the pace of the presentation and made it harder to maintain grader engagement. While the features were working correctly, the live wait times didn't always translate well into a demo setting, especially when we were trying to cover a lot of ground in a limited window. In a few cases, this also introduced risk, since any unexpected slowness or hiccup during the wait could throw off the timing of the rest of the demo.

Going forward, we should be more prepared for these kinds of test cases by having backup materials ready in advance. This could include short pre-recorded videos showing the feature working end-to-end, screenshots of the expected output, or pre-generated results that we can reference if the live version takes too long. The live version should still be the primary way we demonstrate the feature when possible, but having a backup ready means we can keep the demo flowing smoothly if something runs slowly or unexpectedly fails. This approach would help us solidify that the acceptance criteria are met, give us a safety net for time-sensitive moments, and let us spend more of the demo focused on explaining the feature rather than waiting on it.